TCS302 - DATA STRUCTURES WITH C

Bubble Sort

Algorithm, Implementation and Complexity

Unit IV - Lecture 3

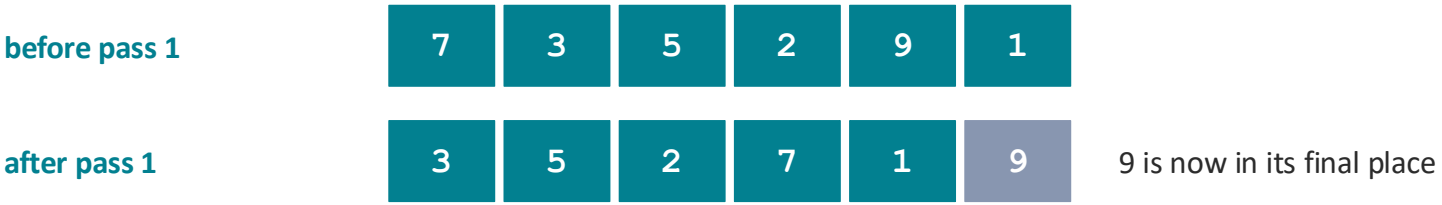
Idea compare neighbours and swap them, so big values bubble to the end

Dry run every pass traced on 7 3 5 2 9 1

Complexity O(n^2) always, or O(n) best case with the swapped flag, verified

The idea: big values bubble up to the end

Compare two neighbours. If the left one is bigger, swap them. Walk from the left end to the right end doing this. The largest value is carried along and ends at the last position, like a bubble rising in water. That is one pass.



The sorted part grows from the RIGHT end. After pass k, the last k values are final, so the next pass can stop one place earlier. With n values, n - 1 passes finish the job.

The algorithm, step by step

1. For i = 0 to n - 2, repeat steps 2 to 5. Each value of i is one pass.
2. Set swapped = 0 at the start of the pass.
3. For j = 0 to n - 2 - i, compare a[j] with a[j + 1].
4. If a[j] > a[j + 1], swap the two and set swapped = 1.
5. If swapped is still 0 after the pass, nothing moved: the array is sorted, so stop early.

Loop invariant
After pass i, the last i + 1 elements are the largest i + 1 values, sitting in their final sorted places. That is why the inner loop can end one place earlier each pass.

Bubble sort in C

```
void bubbleSort(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int swapped = 0;
        for (int j = 0; j < n - 1 - i; j++)
        {
            if (a[j] > a[j + 1])
            {
                int t = a[j];
                a[j] = a[j + 1];
                a[j + 1] = t;
                swapped = 1;
            }
        }
        if (swapped == 0)
            break;   /* sorted */
    }
}
```

Swap, not shift. Neighbours exchange places through the temporary variable t. Insertion sort shifted instead.

n - 1 - i. The last i values are already final, so each pass is one comparison shorter.

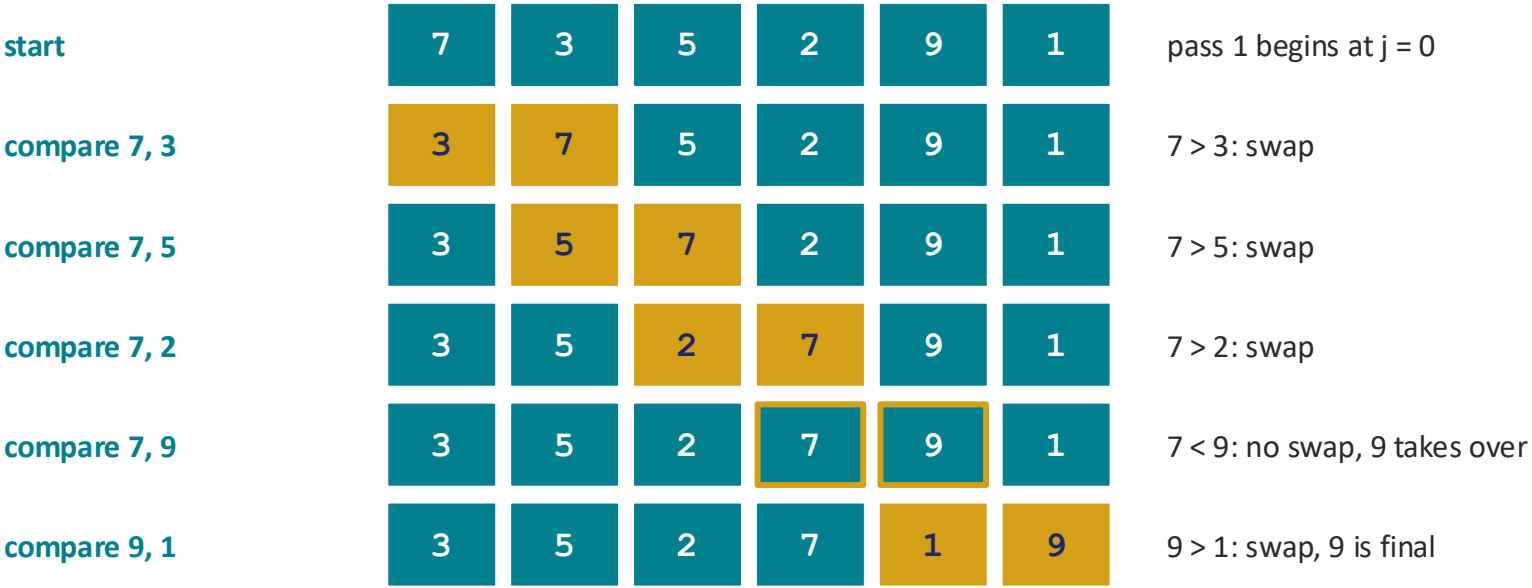
swapped flag. A pass with no swap proves the array is sorted, so the loop can stop.

Dry run on 7 3 5 2 9 1 (verified)

Pass	Array after the pass (unsorted part | final part)	Comparisons	Swaps
start	7 3 5 2 9 1	-	-
1	3 5 2 7 1 | 9	5	4
2	3 2 5 1 | 7 9	4	2
3	2 3 1 | 5 7 9	3	2
4	2 1 | 3 5 7 9	2	1
5	1 | 2 3 5 7 9	1	1
total	1 2 3 5 7 9	15	10

15 comparisons = 5 + 4 + 3 + 2 + 1 = n(n-1)/2. The 10 swaps equal the number of out-of-order pairs in the input - the same 10 moves insertion sort made.

Zoom on pass 1: 7 bubbles up, then 9 takes over



Five comparisons and four swaps - exactly the 5 / 4 recorded for pass 1 in the dry-run table.

Rabbits and turtles: why small values crawl

Rabbit: a large value at the front



5 jumps to the end in ONE pass: 4 comparisons, 4 swaps, and the array is sorted.

Turtle: a small value at the back



1 moves only ONE place left per pass. It needs 4 passes to reach the front (verified).

Large values travel far in one pass, small values crawl one step per pass. Slow turtles are one reason bubble sort does more work than insertion sort (25 vs 18 comparisons on the typical n = 8 input).

Best, worst and average case, measured (n = 8)

Case	Input	Comparisons, plain	Comparisons, with flag	Swaps
Best	`1 2 3 4 5 6 7 8  (already sorted)`	28	7	0
Nearly sorted	`1 2 4 3 5 6 7 8`	28	13	1
Typical	`5 2 8 1 9 3 7 4`	28	25	13
Worst	`8 7 6 5 4 3 2 1  (reversed)`	28	28	28

- Plain version: always n(n-1)/2 = 28 comparisons, whatever the input.
- With the swapped flag the best case falls to n - 1 = 7 comparisons (one pass, no swap): O(n).
- Swaps equal the number of out-of-order pairs. Over all 720 orderings of six values the average is exactly 7.50 = n(n-1)/4 (minimum 0, maximum 15), so the average case is O(n^2).

Quadratic growth: double the data, quadruple the work

n	comparisons, reversed	swaps, reversed	n(n-1)/2	growth from previous n
10	45	45	45	–
20	190	190	190	4.22 times
40	780	780	780	4.11 times

Reversed input
Both comparisons and swaps equal n(n-1)/2, and doubling n multiplies the cost by about 4 - the signature of O(n^2).

Sorted input, n = 40
With the swapped flag: 39 comparisons and 0 swaps. Without the flag the same data costs 780 comparisons.

Properties of bubble sort

Property	Answer	Evidence
Stable	Yes	Strict a[j] > a[j+1] never swaps equal keys: Asha(80) stays before Charu(80)
In-place	Yes	Only i, j, t and swapped are extra: O(1) memory
Adaptive	Only with the flag	Sorted n = 40: 39 comparisons with the flag, 780 without
Time: best / average / worst	O(n) / O(n^2) / O(n^2)	Best case needs the flag; measured on the previous slides

Versus insertion sort. On 7 3 5 2 9 1 both make 10 moves, but bubble sort needs 15 comparisons against 12. On the mixed n = 8 input: 25 comparisons against 18, again with 13 moves each. A swap costs three assignments, a shift only one.

Compare, swap, repeat

- Each pass swaps out-of-order neighbours; after pass i the last i + 1 values are in their final places.
- Dry run on 7 3 5 2 9 1: 15 comparisons and 10 swaps in 5 passes.
- Plain version: always n(n-1)/2 comparisons. With the swapped flag: best O(n) (7 for n = 8), worst and average O(n^2).
- Stable and in-place, but small values move one step per pass, so insertion sort does less work on the same data.

EXIT TICKET Dry-run bubble sort on 9 4 6 2 5. Show the array after each pass and count the total comparisons and swaps.